

BÀI THỰC HÀNH DEEP LEARNING CHO KHOA HỌC DỮ LIỆU

Lab 5. Mạng Hồi Quy: RNN, LSTM, GRU

Hà Minh Tuấn

Khoa Toán - Tin học, ĐHKHTN, ĐHQG-HCM

Ngày 14 tháng 4 năm 2026

Mục tiêu: Sinh viên làm quen với việc xây dựng, huấn luyện, đánh giá và điều chỉnh các mô hình RNN/LSTM/GRU trên dữ liệu chuỗi thời gian thực tế (thời tiết, chứng khoán). Rèn luyện kỹ năng giám sát quá trình huấn luyện, phát hiện và khắc phục các vấn đề thường gặp.

Mục lục

I Kiến Thức Nền Tảng và Thiết Lập Môi Trường	4
1 Giới thiệu tổng quan	4
1.1 Mục tiêu bài thực hành	4
1.2 Kiến thức cần có trước	4
1.3 Tóm tắt lý thuyết	4
2 Thiết lập môi trường	5
2.1 Cài đặt thư viện	5
II Lấy Dữ Liệu Từ API Và Tiền Xử Lý	5
3 Lấy dữ liệu thời tiết từ Open-Meteo API	5
3.1 Giới thiệu API	5
3.2 Code lấy dữ liệu	5
3.3 Khám phá dữ liệu	6
4 Lấy dữ liệu chứng khoán từ Yahoo Finance API	7
4.1 Code lấy dữ liệu	7
4.2 Khám phá dữ liệu chứng khoán	7

5	Tiền xử lý dữ liệu chuỗi thời gian	8
5.1	Chuẩn hóa và tạo cửa sổ trượt	8
5.2	Áp dụng tiền xử lý	9
III	Xây Dựng Và Huấn Luyện Mô Hình	10
6	Định nghĩa mô hình RNN, LSTM, GRU	10
6.1	Kiến trúc chung	10
6.2	So sánh số lượng tham số	12
7	Vòng lặp huấn luyện (Training Loop)	12
7.1	Hàm huấn luyện và đánh giá	12
7.2	Hàm đánh giá trên tập test	13
8	Huấn luyện và so sánh 3 mô hình	14
8.1	Thí nghiệm 1: So sánh RNN, LSTM, GRU trên dữ liệu thời tiết	14
8.2	Thí nghiệm 2: So sánh trên dữ liệu chứng khoán	14
IV	Phân Tích Đường Cong Học Tập	15
9	Vẽ và phân tích Learning Curves	15
9.1	Hàm vẽ đường cong học tập	15
9.2	Vẽ kết quả dự đoán	16
V	Thực Hành Điều Chỉnh Siêu Tham Số	17
10	Ảnh hưởng của Hidden Size	17
11	Ảnh hưởng của Learning Rate	18
12	Ảnh hưởng của Sequence Length	19
13	Ảnh hưởng của số lớp (Num Layers)	20
14	Bảng tổng hợp kết quả thí nghiệm	21
VI	Giám Sát Quá Trình Huấn Luyện Và Khắc Phục Vấn Đề	21
15	Giám sát Gradient: Phát hiện Vanishing và Exploding	21
15.1	Lý thuyết	21
15.2	Code giám sát gradient	22
15.3	Huấn luyện có giám sát gradient	23
16	Kỹ thuật 1: Gradient Clipping	25
16.1	Lý thuyết	25
16.2	Code thực hành	25

17 Kỹ thuật 2: Dropout	27
17.1 Lý thuyết	27
17.2 Code thực hành	27
18 Kỹ thuật 3: Weight Decay (L2 Regularization)	28
18.1 Lý thuyết	28
18.2 Code thực hành	29
19 Kỹ thuật 4: Khởi tạo trọng số (Weight Initialization)	30
19.1 Lý thuyết	30
19.2 Code thực hành	30
20 Kỹ thuật 5: Batch Normalization cho RNN	32
20.1 Lý thuyết	32
20.2 Code thực hành	32
21 Kỹ thuật 6: Early Stopping	33
21.1 Lý thuyết	33
21.2 Code thực hành	33
 VII Tổng Hợp Và Bài Tập Báo Cáo	 35
22 Bảng tổng hợp các kỹ thuật khắc phục	35
23 Bài tập tổng hợp nộp báo cáo	35
23.1 Phần 1: Cơ bản (6 điểm)	35
23.2 Phần 2: Nâng cao (4 điểm)	36
23.3 Hướng dẫn trình bày	36
24 Tài liệu tham khảo	36

Phần I

Kiến Thức Nền Tảng và Thiết Lập Môi Trường

1 Giới thiệu tổng quan

1.1 Mục tiêu bài thực hành

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

1. Hiểu và cài đặt được 3 kiến trúc mạng hồi quy: RNN, LSTM, GRU.
2. Gọi API lấy dữ liệu chuỗi thời gian thực tế (thời tiết, chứng khoán).
3. Tiền xử lý dữ liệu chuỗi thời gian: chuẩn hóa, tạo cửa sổ trượt, chia tập train/val/test.
4. Vẽ và phân tích đường cong học tập (learning curves) để nhận biết overfitting/underfitting.
5. Thực hành điều chỉnh siêu tham số (hyperparameter tuning) và quan sát ảnh hưởng.
6. Sử dụng các công cụ giám sát: phát hiện vanishing/exploding gradients.
7. Áp dụng các kỹ thuật khắc phục: điều chuẩn (regularization), BatchNorm, Dropout, khởi tạo trọng số, gradient clipping.

1.2 Kiến thức cần có trước

Sinh viên cần nắm vững:

- Lập trình Python cơ bản, sử dụng thư viện NumPy, Pandas, Matplotlib.
- Khái niệm cơ bản về mạng nơ-ron: forward pass, backward pass, loss function, optimizer.
- Cơ bản về PyTorch: tensor, autograd, Module, DataLoader.

1.3 Tóm tắt lý thuyết

RNN (Recurrent Neural Network) xử lý dữ liệu tuần tự bằng cách duy trì trạng thái ẩn h_t qua từng bước thời gian:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Hạn chế: RNN gặp vấn đề vanishing/exploding gradient khi chuỗi dài, vì gradient phải lan truyền qua nhiều phép nhân ma trận.

LSTM (Long Short-Term Memory) giải quyết bằng cơ chế cổng (gate):

- **Forget gate** $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$: quyết định thông tin nào cần quên.
- **Input gate** $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$: quyết định thông tin mới nào cần nhớ.
- **Cell state** $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$: bộ nhớ dài hạn.
- **Output gate** $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$: quyết định output.

GRU (Gated Recurrent Unit) đơn giản hóa LSTM, chỉ dùng 2 cổng (reset và update), ít tham số hơn nhưng hiệu quả tương đương trong nhiều bài toán.

2 Thiết lập môi trường

2.1 Cài đặt thư viện

```
# Chạy lệnh này trong terminal hoặc Jupyter Notebook
# !pip install torch numpy pandas matplotlib scikit-learn requests yfinance

import torch
import torch.nn as nn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
import warnings
warnings.filterwarnings('ignore')

# Kiểm tra GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Thiết bị đang sử dụng: {device}")

# Đặt seed để kết quả có thể tái lập
def set_seed(seed=42):
    torch.manual_seed(seed)
    np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)
```

Phần II

Lấy Dữ Liệu Từ API Và Tiền Xử Lý

3 Lấy dữ liệu thời tiết từ Open-Meteo API

3.1 Giới thiệu API

Open-Meteo là API thời tiết miễn phí, không cần API key. Chúng ta sẽ lấy dữ liệu nhiệt độ hàng ngày của TP. Hồ Chí Minh trong 2 năm gần nhất.

3.2 Code lấy dữ liệu

```
import requests

def get_weather_data():
    """
    Lấy dữ liệu nhiệt độ hàng ngày từ Open-Meteo API.
    Trả về DataFrame với cột 'date' và 'temperature'.
    """
```

```

url = "https://archive-api.open-meteo.com/v1/archive"
params = {
    "latitude": 10.82,          # TP. Ho Chi Minh
    "longitude": 106.63,
    "start_date": "2023-01-01",
    "end_date": "2024-12-31",
    "daily": "temperature_2m_mean",
    "timezone": "Asia/Ho_Chi_Minh"
}

response = requests.get(url, params=params)

if response.status_code == 200:
    data = response.json()
    df = pd.DataFrame({
        'date': pd.to_datetime(data['daily']['time']),
        'temperature': data['daily']['temperature_2m_mean']
    })
    print(f"Da lay duoc {len(df)} ngay du lieu thoi tiet")
    print(f"Khoang thoi gian: {df['date'].min()} den {df['date'].max()}")
    print(f"Nhiệt độ: min={df['temperature'].min():.1f}, "
          f"max={df['temperature'].max():.1f}, "
          f"mean={df['temperature'].mean():.1f}")
    return df
else:
    print(f"Loi API: {response.status_code}")
    return None

weather_df = get_weather_data()

```

3.3 Khám phá dữ liệu

```

# Truc quan hoa du lieu thoi tiet
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Bieu do chuoai thoi gian
axes[0].plot(weather_df['date'], weather_df['temperature'],
             color='#2196F3', linewidth=0.8)
axes[0].set_title('Nhiệt độ trung bình hàng ngày - TP.HCM', fontsize=14)
axes[0].set_ylabel('Nhiệt độ (°C)')
axes[0].grid(True, alpha=0.3)

# Bieu do phan phoi
axes[1].hist(weather_df['temperature'], bins=30, color='#FF6B35',
             edgecolor='white', alpha=0.8)
axes[1].set_title('Phân phối nhiệt độ', fontsize=14)
axes[1].set_xlabel('Nhiệt độ (°C)')
axes[1].set_ylabel('Tan suat')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('weather_exploration.png', dpi=150, bbox_inches='tight')
plt.show()

```

Câu hỏi 1: Quan sát biểu đồ, dữ liệu nhiệt độ có tính mùa vụ (seasonality) không? Điều này ảnh hưởng thế nào đến việc chọn độ dài chuỗi đầu vào (sequence length)?

4 Lấy dữ liệu chứng khoán từ Yahoo Finance API

4.1 Code lấy dữ liệu

```
import yfinance as yf

def get_stock_data(ticker="AAPL", period="2y"):
    """
    Lay du lieu gia co phieu tu Yahoo Finance.
    ticker: ma co phieu (VD: AAPL, GOOGL, MSFT, VNM, VIC)
    period: khoang thoi gian (1y, 2y, 5y, max)
    """
    stock = yf.Ticker(ticker)
    df = stock.history(period=period)

    # Giu lai cac cot quan trong
    df = df[['Open', 'High', 'Low', 'Close', 'Volume']].reset_index()
    df.columns = ['date', 'open', 'high', 'low', 'close', 'volume']

    print(f"Ma co phieu: {ticker}")
    print(f"So ngay giao dich: {len(df)}")
    print(f"Khoang thoi gian: {df['date'].min()} den {df['date'].max()}")
    print(f"Gia dong cua: min=${df['close'].min():.2f}, "
          f"max=${df['close'].max():.2f}")

    return df

stock_df = get_stock_data("AAPL", period="2y")
```

4.2 Khám phá dữ liệu chứng khoán

```
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Gia dong cua
axes[0].plot(stock_df['date'], stock_df['close'],
             color='#4CAF50', linewidth=1.0)
axes[0].set_title('Gia dong cua AAPL', fontsize=14)
axes[0].set_ylabel('Gia (USD)')
axes[0].grid(True, alpha=0.3)

# Khoi luong giao dich
axes[1].bar(stock_df['date'], stock_df['volume'],
            color='#FF6B35', alpha=0.6, width=1)
axes[1].set_title('Khoi luong giao dich', fontsize=14)
axes[1].set_ylabel('Volume')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('stock_exploration.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```

Câu hỏi 2: So sánh tính chất của dữ liệu thời tiết và chứng khoán. Dữ liệu nào dễ dự đoán hơn? Tại sao?

5 Tiền xử lý dữ liệu chuỗi thời gian

5.1 Chuẩn hóa và tạo cửa sổ trượt

Đây là bước quan trọng nhất. Chúng ta sẽ biến chuỗi thời gian thành các cặp (input, target) bằng kỹ thuật cửa sổ trượt (sliding window).

Nguyên lý: Với chuỗi $[x_1, x_2, \dots, x_T]$ và window size w :

- Input: $[x_t, x_{t+1}, \dots, x_{t+w-1}]$ (quá khứ)
- Target: x_{t+w} (tương lai cần dự đoán)

```
class TimeSeriesProcessor:
    """
    Lớp tiền xử lý dữ liệu chuỗi thời gian.
    Bao gồm: chuẩn hóa, tạo sliding window, chia train/val/test.
    """
    def __init__(self, sequence_length=30):
        self.seq_len = sequence_length
        self.scaler = MinMaxScaler(feature_range=(0, 1))

    def prepare_data(self, values):
        """
        Chuẩn hóa dữ liệu về khoảng [0, 1].
        values: mảng 1 chiều chứa giá trị chuỗi thời gian.
        """
        values = np.array(values).reshape(-1, 1)
        scaled = self.scaler.fit_transform(values)
        return scaled.flatten()

    def create_sequences(self, data):
        """
        Tạo các cặp (input, target) bằng cửa sổ trượt.
        data: mảng 1 chiều đã chuẩn hóa.
        Trả về: X có shape (num_samples, seq_len, 1)
                y có shape (num_samples,)
        """
        X, y = [], []
        for i in range(len(data) - self.seq_len):
            X.append(data[i : i + self.seq_len])
            y.append(data[i + self.seq_len])

        X = np.array(X).reshape(-1, self.seq_len, 1)
        y = np.array(y)
        return X, y

    def split_data(self, X, y, train_ratio=0.7, val_ratio=0.15):
        """
        Chia dữ liệu theo thứ tự thời gian (KHÔNG shuffle).
        """
```


*Ly do: du lieu chuoai thoi gian co tinh thu tu,
neu shuffle se gay data leakage.*

*/--- 70% train ---/--- 15% val ---/--- 15% test ---/
"""*

```
n = len(X)
train_end = int(n * train_ratio)
val_end = int(n * (train_ratio + val_ratio))

X_train, y_train = X[:train_end], y[:train_end]
X_val, y_val = X[train_end:val_end], y[train_end:val_end]
X_test, y_test = X[val_end:], y[val_end:]

print(f"Train: {len(X_train)} mau")
print(f"Val: {len(X_val)} mau")
print(f"Test: {len(X_test)} mau")

return (X_train, y_train), (X_val, y_val), (X_test, y_test)

def to_tensors(self, X, y):
    """Chuyen numpy array sang PyTorch tensor."""
    X_tensor = torch.FloatTensor(X).to(device)
    y_tensor = torch.FloatTensor(y).to(device)
    return X_tensor, y_tensor

def inverse_transform(self, values):
    """Chuyen gia tri da chuan hoa ve gia tri goc."""
    return self.scaler.inverse_transform(
        values.reshape(-1, 1)
    ).flatten()
```

5.2 Áp dụng tiền xử lý

```
# === XU LY DU LIEU THOI TIET ===
print("=" * 50)
print("DU LIEU THOI TIET")
print("=" * 50)

weather_processor = TimeSeriesProcessor(sequence_length=30)
weather_scaled = weather_processor.prepare_data(weather_df['temperature'].values)
X_weather, y_weather = weather_processor.create_sequences(weather_scaled)
weather_splits = weather_processor.split_data(X_weather, y_weather)

(X_train_w, y_train_w), (X_val_w, y_val_w), (X_test_w, y_test_w) = weather_splits

# Chuyen sang tensor
X_train_w, y_train_w = weather_processor.to_tensors(X_train_w, y_train_w)
X_val_w, y_val_w = weather_processor.to_tensors(X_val_w, y_val_w)
X_test_w, y_test_w = weather_processor.to_tensors(X_test_w, y_test_w)

print(f"\nShape: X_train={X_train_w.shape}, y_train={y_train_w.shape}")

# === XU LY DU LIEU CHUNG KHOAN ===
```

```

print("\n" + "=" * 50)
print("DU LIEU CHUNG KHOAN")
print("=" * 50)

stock_processor = TimeSeriesProcessor(sequence_length=30)
stock_scaled = stock_processor.prepare_data(stock_df['close'].values)
X_stock, y_stock = stock_processor.create_sequences(stock_scaled)
stock_splits = stock_processor.split_data(X_stock, y_stock)

(X_train_s, y_train_s), (X_val_s, y_val_s), (X_test_s, y_test_s) = stock_splits

X_train_s, y_train_s = stock_processor.to_tensors(X_train_s, y_train_s)
X_val_s, y_val_s = stock_processor.to_tensors(X_val_s, y_val_s)
X_test_s, y_test_s = stock_processor.to_tensors(X_test_s, y_test_s)

```

Câu hỏi 3: Tại sao khi chia dữ liệu chuỗi thời gian, chúng ta KHÔNG được shuffle? Nếu shuffle thì sẽ xảy ra vấn đề gì?

Câu hỏi 4: Nếu thay đổi `sequence_length` từ 30 thành 7 hoặc 60, bạn nghĩ kết quả sẽ thay đổi thế nào? (Sẽ thí nghiệm ở phần sau.)

Phần III

Xây Dựng Và Huấn Luyện Mô Hình

6 Định nghĩa mô hình RNN, LSTM, GRU

6.1 Kiến trúc chung

Cả 3 mô hình đều có kiến trúc tương tự: lớp recurrent → lớp fully connected. Chúng ta thiết kế một class chung, chỉ thay đổi loại cell.

```

class RecurrentModel(nn.Module):
    """
    Mô hình hồi quy tổng quát cho dữ liệu chuỗi thời gian.

    Tham số:
        model_type: 'RNN', 'LSTM', hoặc 'GRU'
        input_size: số features đầu vào (=1 vì chỉ có 1 giá trị)
        hidden_size: số neuron trong lớp ẩn
        num_layers: số lớp recurrent chồng lên nhau
        dropout: tỉ lệ dropout giữa các lớp (chỉ áp dụng khi num_layers > 1)
    """
    def __init__(self, model_type='LSTM', input_size=1,
                  hidden_size=64, num_layers=2, dropout=0.0):
        super(RecurrentModel, self).__init__()

        self.model_type = model_type
        self.hidden_size = hidden_size
        self.num_layers = num_layers

        # Chọn loại cell tương ứng
        if model_type == 'RNN':

```

```

        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,      # Input: (batch, seq_len, features)
            dropout=dropout if num_layers > 1 else 0
        )
    elif model_type == 'LSTM':
        self.rnn = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0
        )
    elif model_type == 'GRU':
        self.rnn = nn.GRU(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0
        )
    else:
        raise ValueError(f"model_type phải là 'RNN', 'LSTM', hoặc 'GRU'")

    # Lớp fully connected để dự đoán giá trị
    self.fc = nn.Linear(hidden_size, 1)

def forward(self, x):
    """
    x: tensor có shape (batch_size, seq_len, input_size)
    Tra về: tensor có shape (batch_size,) là giá trị dự đoán
    """
    # rnn_out: (batch, seq_len, hidden_size) - output tại mỗi bước
    # hidden: trạng thái ẩn cuối cùng
    if self.model_type == 'LSTM':
        rnn_out, (hidden, cell) = self.rnn(x)
    else:
        rnn_out, hidden = self.rnn(x)

    # Chỉ lấy output tại bước thời gian cuối cùng
    last_output = rnn_out[:, -1, :] # (batch, hidden_size)

    # Dự đoán giá trị tiếp theo
    prediction = self.fc(last_output) # (batch, 1)
    return prediction.squeeze(-1)     # (batch,)

def count_parameters(self):
    """Đếm tổng số tham số của mô hình."""
    total = sum(p.numel() for p in self.parameters() if p.requires_grad)
    return total

```

6.2 So sánh số lượng tham số

```
# So sánh số tham số giữa 3 mô hình
print(f"{'Mo hình':<10} {'Số tham số':>12}")
print("-" * 25)

for model_type in ['RNN', 'LSTM', 'GRU']:
    model = RecurrentModel(model_type=model_type, hidden_size=64, num_layers=2)
    params = model.count_parameters()
    print(f"{'model_type':<10} {'params':>12,}")

# Kết quả sẽ cho thấy:
# LSTM có nhiều tham số nhất (~4x RNN) vì có 4 cổng
# GRU nằm giữa (~3x RNN) vì có 3 cổng
# RNN có ít tham số nhất
```

Câu hỏi 5: Tại sao LSTM có nhiều tham số gấp khoảng 4 lần RNN? Liên hệ với công thức của các cổng (gate) ở phần lý thuyết.

7 Vòng lặp huấn luyện (Training Loop)

7.1 Hàm huấn luyện và đánh giá

```
def train_model(model, X_train, y_train, X_val, y_val,
                num_epochs=100, learning_rate=0.001, batch_size=32,
                verbose=True):
    """
    Huấn luyện mô hình và ghi lại lịch sử loss.

    Trả về:
        history: dict chứa 'train_loss' và 'val_loss' theo từng epoch
    """
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    history = {'train_loss': [], 'val_loss': []}

    # Tạo DataLoader cho mini-batch training
    train_dataset = torch.utils.data.TensorDataset(X_train, y_train)
    train_loader = torch.utils.data.DataLoader(
        train_dataset, batch_size=batch_size, shuffle=False
    )

    for epoch in range(num_epochs):
        # === TRAINING ===
        model.train()
        epoch_train_loss = 0.0
        num_batches = 0

        for batch_X, batch_y in train_loader:
            optimizer.zero_grad()
            predictions = model(batch_X)
            loss = criterion(predictions, batch_y)
```

```

        loss.backward()
        optimizer.step()

        epoch_train_loss += loss.item()
        num_batches += 1

    avg_train_loss = epoch_train_loss / num_batches

    # === VALIDATION ===
    model.eval()
    with torch.no_grad():
        val_predictions = model(X_val)
        val_loss = criterion(val_predictions, y_val).item()

    history['train_loss'].append(avg_train_loss)
    history['val_loss'].append(val_loss)

    if verbose and (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1:3d}/{num_epochs} | "
              f"Train Loss: {avg_train_loss:.6f} | "
              f"Val Loss: {val_loss:.6f}")

    return history

```

7.2 Hàm đánh giá trên tập test

```

def evaluate_model(model, X_test, y_test, processor):
    """
    Danh gia mo hinh tren tap test.
    Tra ve gia tri thuc va gia tri du doan (da inverse transform).
    """
    model.eval()
    with torch.no_grad():
        predictions = model(X_test).cpu().numpy()

    y_true = y_test.cpu().numpy()

    # Chuyen ve gia tri goc
    predictions_orig = processor.inverse_transform(predictions)
    y_true_orig = processor.inverse_transform(y_true)

    # Tinh cac chi so
    mse = mean_squared_error(y_true_orig, predictions_orig)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true_orig, predictions_orig)

    print(f"MSE: {mse:.4f}")
    print(f"RMSE: {rmse:.4f}")
    print(f"MAE: {mae:.4f}")

    return y_true_orig, predictions_orig

```

8 Huấn luyện và so sánh 3 mô hình

8.1 Thí nghiệm 1: So sánh RNN, LSTM, GRU trên dữ liệu thời tiết

```
# Cấu hình chung cho thí nghiệm
CONFIG = {
    'hidden_size': 64,
    'num_layers': 2,
    'num_epochs': 100,
    'learning_rate': 0.001,
    'batch_size': 32,
}

results_weather = {}

for model_type in ['RNN', 'LSTM', 'GRU']:
    print(f"\n{'='*50}")
    print(f"  Huấn luyện {model_type} trên dữ liệu THỜI TIẾT")
    print(f"{'='*50}")

    set_seed(42)

    model = RecurrentModel(
        model_type=model_type,
        hidden_size=CONFIG['hidden_size'],
        num_layers=CONFIG['num_layers']
    ).to(device)

    print(f"So sánh số: {model.count_parameters():,}")

    history = train_model(
        model, X_train_w, y_train_w, X_val_w, y_val_w,
        num_epochs=CONFIG['num_epochs'],
        learning_rate=CONFIG['learning_rate'],
        batch_size=CONFIG['batch_size']
    )

    print("\nĐánh giá trên tập test:")
    y_true, y_pred = evaluate_model(model, X_test_w, y_test_w, weather_processor)

    results_weather[model_type] = {
        'history': history,
        'y_true': y_true,
        'y_pred': y_pred,
        'model': model
    }
```

8.2 Thí nghiệm 2: So sánh trên dữ liệu chứng khoán

```
results_stock = {}

for model_type in ['RNN', 'LSTM', 'GRU']:
```

```

print(f"\n{'='*50}")
print(f"  Huan luyen {model_type} tren du lieu CHUNG KHOAN")
print(f"{'='*50}")

set_seed(42)

model = RecurrentModel(
    model_type=model_type,
    hidden_size=CONFIG['hidden_size'],
    num_layers=CONFIG['num_layers']
).to(device)

history = train_model(
    model, X_train_s, y_train_s, X_val_s, y_val_s,
    num_epochs=CONFIG['num_epochs'],
    learning_rate=CONFIG['learning_rate'],
    batch_size=CONFIG['batch_size']
)

print("\nDanh gia tren tap test:")
y_true, y_pred = evaluate_model(model, X_test_s, y_test_s, stock_processor)

results_stock[model_type] = {
    'history': history,
    'y_true': y_true,
    'y_pred': y_pred,
    'model': model
}

```

Phần IV

Phân Tích Đường Cong Học Tập

9 Vẽ và phân tích Learning Curves

9.1 Hàm vẽ đường cong học tập

Đường cong học tập (learning curve) là công cụ quan trọng nhất để chẩn đoán mô hình:

- **Overfitting:** Train loss giảm nhưng val loss tăng — khoảng cách giữa 2 đường ngày càng lớn.
- **Underfitting:** Cả train loss và val loss đều cao, giảm chậm — mô hình chưa đủ phức tạp.
- **Good fit:** Cả 2 đường giảm song song, khoảng cách nhỏ và ổn định.

```

def plot_learning_curves(results, dataset_name):
    """
    Vẽ đường cong học tập cho cả 3 mô hình trên cùng 1 figure.
    """
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

```

```

for idx, model_type in enumerate(['RNN', 'LSTM', 'GRU']):
    ax = axes[idx]
    history = results[model_type]['history']
    epochs = range(1, len(history['train_loss']) + 1)

    ax.plot(epochs, history['train_loss'],
            label='Train Loss', color='#2196F3', linewidth=2)
    ax.plot(epochs, history['val_loss'],
            label='Val Loss', color='#F44336', linewidth=2, linestyle='--')

    ax.set_title(f'{model_type} - {dataset_name}', fontsize=14,
        ↪ fontweight='bold')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss (MSE)')
    ax.legend(fontsize=11)
    ax.grid(True, alpha=0.3)

    # Danh dau vung overfitting (neu co)
    train_losses = history['train_loss']
    val_losses = history['val_loss']

    # Tim diem val loss bat dau tang
    min_val_idx = np.argmin(val_losses)
    if min_val_idx < len(val_losses) - 10:
        ax.axvline(x=min_val_idx + 1, color='orange',
            linestyle=':', linewidth=1.5, alpha=0.7)
        ax.annotate('Val loss min',
            xy=(min_val_idx + 1, val_losses[min_val_idx]),
            xytext=(min_val_idx + 15, val_losses[min_val_idx] * 1.5),
            arrowprops=dict(arrowstyle='->', color='orange'),
            fontsize=9, color='orange')

plt.suptitle(f'Duong cong hoc tap - {dataset_name}',
    fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(f'learning_curves_{dataset_name}.png', dpi=150,
    ↪ bbox_inches='tight')
plt.show()

# Ve cho 2 bo du lieu
plot_learning_curves(results_weather, "Thoi Tiet")
plot_learning_curves(results_stock, "Chung Khoan")

```

9.2 Vẽ kết quả dự đoán

```

def plot_predictions(results, dataset_name):
    """Ve gia tri thuc vs du doan cho 3 mo hinh."""
    fig, axes = plt.subplots(3, 1, figsize=(14, 12))

    for idx, model_type in enumerate(['RNN', 'LSTM', 'GRU']):
        ax = axes[idx]
        y_true = results[model_type]['y_true']

```



```

y_pred = results[model_type]['y_pred']

ax.plot(y_true, label='Gia tri thuc', color='#2196F3', linewidth=1.5)
ax.plot(y_pred, label='Du doan', color='#F44336',
        linewidth=1.5, linestyle='--', alpha=0.8)

rmse = np.sqrt(mean_squared_error(y_true, y_pred))
ax.set_title(f'{model_type} | RMSE = {rmse:.4f}',
             fontsize=13, fontweight='bold')
ax.set_ylabel('Gia tri')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

axes[-1].set_xlabel('Buoc thoi gian')
plt.suptitle(f'Du doan vs Thuc te - {dataset_name}',
            fontsize=15, fontweight='bold')
plt.tight_layout()
plt.savefig(f'predictions_{dataset_name}.png', dpi=150, bbox_inches='tight')
plt.show()

plot_predictions(results_weather, "Thoi Tiet")
plot_predictions(results_stock, "Chung Khoan")

```

Bài tập 1: Quan sát các đường cong học tập và trả lời:

1. Mô hình nào có dấu hiệu overfitting rõ nhất? Bạn nhận biết qua đặc điểm gì?
2. Mô hình nào cho RMSE thấp nhất trên dữ liệu thời tiết? Trên chứng khoán?
3. Tại sao kết quả dự đoán chứng khoán thường kém hơn thời tiết?

Phần V

Thực Hành Điều Chỉnh Siêu Tham Số

10 Ảnh hưởng của Hidden Size

```

# Thi nghiem: thay doi hidden_size va quan sat
hidden_sizes = [16, 32, 64, 128, 256]
hidden_results = {}

for hs in hidden_sizes:
    print(f"\n--- Hidden size = {hs} ---")
    set_seed(42)

    model = RecurrentModel(
        model_type='LSTM', hidden_size=hs, num_layers=2
    ).to(device)

    print(f"So tham so: {model.count_parameters():,}")

    history = train_model(
        model, X_train_w, y_train_w, X_val_w, y_val_w,

```

```

        num_epochs=100, learning_rate=0.001, batch_size=32,
        verbose=False
    )

    hidden_results[hs] = history
    print(f"Train loss cuoi: {history['train_loss'][-1]:.6f} | "
          f"Val loss cuoi: {history['val_loss'][-1]:.6f}")

# Ve bieu do so sanh
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for hs in hidden_sizes:
    axes[0].plot(hidden_results[hs]['train_loss'], label=f'hs={hs}')
    axes[1].plot(hidden_results[hs]['val_loss'], label=f'hs={hs}')

axes[0].set_title('Train Loss theo Hidden Size', fontsize=13)
axes[1].set_title('Val Loss theo Hidden Size', fontsize=13)

for ax in axes:
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.legend()
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('hidden_size_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Bài tập 2: Nhận xét:

1. Hidden size nào cho val loss thấp nhất?
2. Hidden size quá lớn (256) có vấn đề gì? (Gợi ý: quan sát khoảng cách train-val loss.)
3. Hidden size quá nhỏ (16) có vấn đề gì? (Gợi ý: train loss có giảm đủ thấp không?)

11 Ảnh hưởng của Learning Rate

```

learning_rates = [0.01, 0.005, 0.001, 0.0005, 0.0001]
lr_results = {}

for lr in learning_rates:
    print(f"\n--- Learning rate = {lr} ---")
    set_seed(42)

    model = RecurrentModel(
        model_type='LSTM', hidden_size=64, num_layers=2
    ).to(device)

    history = train_model(
        model, X_train_w, y_train_w, X_val_w, y_val_w,
        num_epochs=100, learning_rate=lr, batch_size=32,
        verbose=False
    )

```

```

lr_results[lr] = history
print(f"Train loss cuoi: {history['train_loss'][-1]:.6f} | "
      f"Val loss cuoi: {history['val_loss'][-1]:.6f}")

# Ve bieu do
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for lr in learning_rates:
    axes[0].plot(lr_results[lr]['train_loss'], label=f'lr={lr}')
    axes[1].plot(lr_results[lr]['val_loss'], label=f'lr={lr}')

axes[0].set_title('Train Loss theo Learning Rate', fontsize=13)
axes[1].set_title('Val Loss theo Learning Rate', fontsize=13)
axes[0].set_yscale('log')
axes[1].set_yscale('log')

for ax in axes:
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss (log scale)')
    ax.legend()
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('learning_rate_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Bài tập 3: Nhận xét:

1. LR = 0.01 có hiện tượng gì? (Loss có dao động mạnh hay NaN không?)
2. LR = 0.0001 có vấn đề gì? (Mô hình đã hội tụ sau 100 epochs chưa?)
3. LR nào cho kết quả tốt nhất? Tại sao?

12 Ảnh hưởng của Sequence Length

```

seq_lengths = [7, 14, 30, 60, 90]
seq_results = {}

for seq_len in seq_lengths:
    print(f"\n--- Sequence length = {seq_len} ---")
    set_seed(42)

    # Tien xu ly lai voi seq_len moi
    proc = TimeSeriesProcessor(sequence_length=seq_len)
    scaled = proc.prepare_data(weather_df['temperature'].values)
    X, y = proc.create_sequences(scaled)
    (Xtr, ytr), (Xv, yv), (Xte, yte) = proc.split_data(X, y)
    Xtr, ytr = proc.to_tensors(Xtr, ytr)
    Xv, yv = proc.to_tensors(Xv, yv)

    model = RecurrentModel(
        model_type='LSTM', hidden_size=64, num_layers=2
    )

```

```

).to(device)

history = train_model(
    model, Xtr, ytr, Xv, yv,
    num_epochs=100, learning_rate=0.001, batch_size=32,
    verbose=False
)

seq_results[seq_len] = history
print(f"Val loss cuoi: {history['val_loss'][-1]:.6f}")

# Ve bieu do
fig, ax = plt.subplots(figsize=(10, 5))

for seq_len in seq_lengths:
    ax.plot(seq_results[seq_len]['val_loss'], label=f'seq_len={seq_len}')

ax.set_title('Val Loss theo Sequence Length', fontsize=14)
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('seq_length_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Bài tập 4: Thực hiện tương tự với dữ liệu chứng khoán. Sequence length nào tốt nhất cho mỗi loại dữ liệu? Giải thích tại sao.

13 Ảnh hưởng của số lớp (Num Layers)

```

num_layers_list = [1, 2, 3, 4]
layer_results = {}

for nl in num_layers_list:
    print(f"\n--- Num layers = {nl} ---")
    set_seed(42)

    model = RecurrentModel(
        model_type='LSTM', hidden_size=64, num_layers=nl
    ).to(device)

    print(f"So tham so: {model.count_parameters():,}")

    history = train_model(
        model, X_train_w, y_train_w, X_val_w, y_val_w,
        num_epochs=100, learning_rate=0.001, batch_size=32,
        verbose=False
    )

    layer_results[nl] = history
    print(f"Train loss: {history['train_loss'][-1]:.6f} | ")

```

```
f"Val loss: {history['val_loss'][-1]:.6f}")
```

Bài tập 5: Vẽ biểu đồ so sánh tương tự các thí nghiệm trước. Nhận xét: tăng số lớp có luôn tốt hơn không? Khi nào thì “sâu hơn” lại cho kết quả kém hơn?

14 Bảng tổng hợp kết quả thí nghiệm

Bài tập 6: Điền vào bảng sau sau khi chạy tất cả thí nghiệm:

Siêu tham số	Giá trị	Train Loss	Val Loss	Nhận xét
Model type	RNN			
	LSTM			
	GRU			
Hidden size	16			
	64			
	256			
Learning rate	0.01			
	0.001			
	0.0001			
Seq length	7			
	30			
	90			

Phần VI

Giám Sát Quá Trình Huấn Luyện Và Khắc Phục Vấn Đề

15 Giám sát Gradient: Phát hiện Vanishing và Exploding

15.1 Lý thuyết

Trong mạng hồi quy, gradient lan truyền ngược qua T bước thời gian. Theo chain rule:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L}{\partial h_T} \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}} \cdot \frac{\partial h_t}{\partial W}$$

Nếu $\left\| \frac{\partial h_k}{\partial h_{k-1}} \right\| < 1$ liên tục thì tích sẽ tiến về 0 (**vanishing**). Nếu > 1 liên tục thì tích phát tán (**exploding**).

15.2 Code giám sát gradient

```
class GradientMonitor:
    """
    Giám sát gradient trong quá trình huấn luyện.
    Ghi lại norm, max, min của gradient mỗi layer tại mỗi epoch.
    """
    def __init__(self):
        self.grad_norms = {}      # {layer_name: [norm_epoch1, norm_epoch2, ...]}
        self.grad_maxs = {}

    def record(self, model):
        """Ghi lại thông kê gradient của tất cả các layer."""
        for name, param in model.named_parameters():
            if param.grad is not None:
                grad_norm = param.grad.data.norm(2).item()
                grad_max = param.grad.data.abs().max().item()

                if name not in self.grad_norms:
                    self.grad_norms[name] = []
                    self.grad_maxs[name] = []

                self.grad_norms[name].append(grad_norm)
                self.grad_maxs[name].append(grad_max)

    def check_health(self, model):
        """Kiểm tra sức khỏe gradient ngay lập tức."""
        issues = []
        for name, param in model.named_parameters():
            if param.grad is not None:
                g = param.grad.data
                norm = g.norm(2).item()

                if g.isnan().any():
                    issues.append(f" NaN gradient: {name}")
                elif g.isinf().any():
                    issues.append(f" Inf gradient: {name}")
                elif norm > 100:
                    issues.append(f" Exploding (norm={norm:.1f}): {name}")
                elif norm < 1e-7:
                    issues.append(f" Vanishing (norm={norm:.2e}): {name}")

        return issues

    def plot(self, title="Gradient Norms"):
        """Vẽ biểu đồ gradient norm theo thời gian."""
        fig, ax = plt.subplots(figsize=(12, 5))

        for name, norms in self.grad_norms.items():
            # Chỉ vẽ các layer chính (bỏ qua bias để giảm nhiễu)
            if 'weight' in name:
                label = name.replace('rnn.', '').replace('_1', ' L')
                ax.plot(norms, label=label, alpha=0.8)
```

```

ax.set_title(title, fontsize=14)
ax.set_xlabel('Step')
ax.set_ylabel('Gradient L2 Norm')
ax.set_yscale('log')
ax.legend(fontsize=8, loc='upper right', ncol=2)
ax.grid(True, alpha=0.3)

# Ve nguong canh bao
ax.axhline(y=100, color='red', linestyle=':', alpha=0.5, label='Exploding
→ threshold')
ax.axhline(y=1e-7, color='blue', linestyle=':', alpha=0.5,
→ label='Vanishing threshold')

plt.tight_layout()
plt.savefig(f'gradient_monitor.png', dpi=150, bbox_inches='tight')
plt.show()

```

15.3 Huấn luyện có giám sát gradient

```

def train_with_monitoring(model, X_train, y_train, X_val, y_val,
                           num_epochs=100, learning_rate=0.001, batch_size=32):
    """
    Huan luyen voi giam sat gradient day du.
    """
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    monitor = GradientMonitor()

    history = {'train_loss': [], 'val_loss': []}

    train_dataset = torch.utils.data.TensorDataset(X_train, y_train)
    train_loader = torch.utils.data.DataLoader(
        train_dataset, batch_size=batch_size, shuffle=False
    )

    for epoch in range(num_epochs):
        model.train()
        epoch_loss = 0.0
        num_batches = 0

        for batch_X, batch_y in train_loader:
            optimizer.zero_grad()
            predictions = model(batch_X)
            loss = criterion(predictions, batch_y)
            loss.backward()

            # === GIAM SAT GRADIENT ===
            monitor.record(model)
            issues = monitor.check_health(model)
            if issues and epoch % 20 == 0:
                print(f"Epoch {epoch+1} - Van de gradient:")
                for issue in issues:
                    print(issue)

```

```

        optimizer.step()
        epoch_loss += loss.item()
        num_batches += 1

    avg_train_loss = epoch_loss / num_batches

    model.eval()
    with torch.no_grad():
        val_pred = model(X_val)
        val_loss = criterion(val_pred, y_val).item()

    history['train_loss'].append(avg_train_loss)
    history['val_loss'].append(val_loss)

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1:3d} | Train: {avg_train_loss:.6f} "
              f"| Val: {val_loss:.6f}")

    return history, monitor

# === THI NGHIEM: RNN thuan (de thay vanishing gradient) ===
print("=" * 60)
print("  THI NGHIEM: RNN voi chuoi dai (de thay vanishing gradient)")
print("=" * 60)

# Tao du lieu voi sequence_length lon
proc_long = TimeSeriesProcessor(sequence_length=90)
scaled_long = proc_long.prepare_data(weather_df['temperature'].values)
X_long, y_long = proc_long.create_sequences(scaled_long)
(Xtr_l, ytr_l), (Xv_l, yv_l), _ = proc_long.split_data(X_long, y_long)
Xtr_l, ytr_l = proc_long.to_tensors(Xtr_l, ytr_l)
Xv_l, yv_l = proc_long.to_tensors(Xv_l, yv_l)

set_seed(42)
rnn_model = RecurrentModel(
    model_type='RNN', hidden_size=64, num_layers=3
).to(device)

rnn_history, rnn_monitor = train_with_monitoring(
    rnn_model, Xtr_l, ytr_l, Xv_l, yv_l,
    num_epochs=50, learning_rate=0.001
)

rnn_monitor.plot("Gradient Norms - RNN voi chuoi dai (seq=90)")

# === SO SANH: LSTM voi cung cau hinh ===
print("\n" + "=" * 60)
print("  SO SANH: LSTM voi cung cau hinh")
print("=" * 60)

set_seed(42)
lstm_model = RecurrentModel(
    model_type='LSTM', hidden_size=64, num_layers=3

```



```

).to(device)

lstm_history, lstm_monitor = train_with_monitoring(
    lstm_model, Xtr_l, ytr_l, Xv_l, yv_l,
    num_epochs=50, learning_rate=0.001
)

lstm_monitor.plot("Gradient Norms - LSTM voi chuoi dai (seq=90)")

```

Bài tập 7: So sánh biểu đồ gradient norm giữa RNN và LSTM:

1. RNN có layer nào bị vanishing gradient (norm $< 10^{-7}$) không?
2. LSTM có giải quyết được vấn đề đó không? Gradient norm ổn định hơn không?
3. Thử thêm GRU vào so sánh.

16 Kỹ thuật 1: Gradient Clipping

16.1 Lý thuyết

Gradient clipping giới hạn norm của gradient, ngăn exploding gradient:

$$g \leftarrow g \cdot \frac{\max_norm}{\max(\|g\|, \max_norm)}$$

Nếu $\|g\| \leq \max_norm$ thì gradient giữ nguyên. Nếu $\|g\| > \max_norm$ thì gradient bị co lại.

16.2 Code thực hành

```

def train_with_clipping(model, X_train, y_train, X_val, y_val,
                        num_epochs=100, learning_rate=0.001, batch_size=32,
                        max_grad_norm=None):
    """
    Huan luyen voi gradient clipping.
    max_grad_norm: ngưỡng clip (None = không clip)
    """
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    history = {'train_loss': [], 'val_loss': [], 'grad_norms': []}

    train_dataset = torch.utils.data.TensorDataset(X_train, y_train)
    train_loader = torch.utils.data.DataLoader(
        train_dataset, batch_size=batch_size, shuffle=False
    )

    for epoch in range(num_epochs):
        model.train()
        epoch_loss = 0.0
        epoch_grad_norm = 0.0
        num_batches = 0

```

```

for batch_X, batch_y in train_loader:
    optimizer.zero_grad()
    predictions = model(batch_X)
    loss = criterion(predictions, batch_y)
    loss.backward()

    # Tinh grad norm TRUOC khi clip
    total_norm = 0
    for p in model.parameters():
        if p.grad is not None:
            total_norm += p.grad.data.norm(2).item() ** 2
    total_norm = total_norm ** 0.5
    epoch_grad_norm += total_norm

    # === GRADIENT CLIPPING ===
    if max_grad_norm is not None:
        torch.nn.utils.clip_grad_norm_(
            model.parameters(), max_grad_norm
        )

    optimizer.step()
    epoch_loss += loss.item()
    num_batches += 1

history['train_loss'].append(epoch_loss / num_batches)
history['grad_norms'].append(epoch_grad_norm / num_batches)

model.eval()
with torch.no_grad():
    val_pred = model(X_val)
    val_loss = criterion(val_pred, y_val).item()
    history['val_loss'].append(val_loss)

return history

# === THI NGHIEM: So sanh co va khong co gradient clipping ===
clip_values = [None, 5.0, 1.0, 0.5]
clip_results = {}

for clip in clip_values:
    label = f"clip={clip}" if clip else "No clipping"
    print(f"\n--- {label} ---")
    set_seed(42)

    model = RecurrentModel(
        model_type='RNN', hidden_size=64, num_layers=3
    ).to(device)

    history = train_with_clipping(
        model, X_train_w, y_train_w, X_val_w, y_val_w,
        num_epochs=100, learning_rate=0.005, # LR lon de de thay exploding
        max_grad_norm=clip
    )

```

```

clip_results[label] = history
print(f"Val loss cuoi: {history['val_loss'][-1]:.6f}")

# Ve bieu do so sanh
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for label, history in clip_results.items():
    axes[0].plot(history['val_loss'], label=label)
    axes[1].plot(history['grad_norms'], label=label)

axes[0].set_title('Val Loss', fontsize=13)
axes[0].set_ylabel('Loss')
axes[1].set_title('Gradient Norm (truoc clip)', fontsize=13)
axes[1].set_ylabel('Norm')
axes[1].set_yscale('log')

for ax in axes:
    ax.set_xlabel('Epoch')
    ax.legend()
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('gradient_clipping.png', dpi=150, bbox_inches='tight')
plt.show()

```

Bài tập 8: Nhận xét về ảnh hưởng của gradient clipping. Giá trị max_grad_norm nào cho kết quả ổn định nhất?

17 Kỹ thuật 2: Dropout

17.1 Lý thuyết

Dropout ngẫu nhiên “tắt” một tỉ lệ p các neuron trong quá trình training, buộc mạng học các đặc trưng robust hơn, giảm overfitting:

$$\tilde{h}_i = \begin{cases} 0 & \text{với xác suất } p \\ h_i/(1-p) & \text{với xác suất } 1-p \end{cases}$$

17.2 Code thực hành

```

# === THI NGHIEM: Anh huong cua Dropout ===
dropout_rates = [0.0, 0.1, 0.2, 0.3, 0.5]
dropout_results = {}

for dr in dropout_rates:
    print(f"\n--- Dropout = {dr} ---")
    set_seed(42)

    model = RecurrentModel(
        model_type='LSTM',
        hidden_size=128,      # Hidden lon de de thay overfitting
        num_layers=3,
        dropout=dr
    )

```

```

).to(device)

history = train_model(
    model, X_train_w, y_train_w, X_val_w, y_val_w,
    num_epochs=150,      # Train lâu để thay overfitting
    learning_rate=0.001, batch_size=32,
    verbose=False
)

dropout_results[dr] = history
gap = history['train_loss'][-1] - history['val_loss'][-1]
print(f"Train: {history['train_loss'][-1]:.6f} | "
      f"Val: {history['val_loss'][-1]:.6f} | "
      f"Gap: {abs(gap):.6f}")

# Vẽ biểu đồ
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for dr in dropout_rates:
    axes[0].plot(dropout_results[dr]['train_loss'], label=f'drop={dr}')
    axes[1].plot(dropout_results[dr]['val_loss'], label=f'drop={dr}')

axes[0].set_title('Train Loss theo Dropout', fontsize=13)
axes[1].set_title('Val Loss theo Dropout', fontsize=13)

for ax in axes:
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.legend()
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('dropout_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Bài tập 9:

1. Dropout = 0.0 có overfitting không? (So sánh train loss và val loss.)
2. Dropout rate nào giảm overfitting tốt nhất mà không gây underfitting?
3. Dropout = 0.5 có quá mạnh không? Quan sát train loss.

18 Kỹ thuật 3: Weight Decay (L2 Regularization)

18.1 Lý thuyết

Weight decay thêm penalty vào loss function để giới hạn độ lớn của trọng số:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_i w_i^2$$

Trong đó λ là hệ số điều chuẩn. Giá trị λ lớn $\rightarrow$ trọng số bị ép nhỏ $\rightarrow$ mô hình đơn giản hơn $\rightarrow$ giảm overfitting.

18.2 Code thực hành

```
weight_decays = [0, 1e-5, 1e-4, 1e-3, 1e-2]
wd_results = {}

for wd in weight_decays:
    print(f"\n--- Weight Decay = {wd} ---")
    set_seed(42)

    model = RecurrentModel(
        model_type='LSTM', hidden_size=128, num_layers=3
    ).to(device)

    # Weight decay duoc truyen vao optimizer
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(
        model.parameters(), lr=0.001, weight_decay=wd
    )

    history = {'train_loss': [], 'val_loss': []}
    train_dataset = torch.utils.data.TensorDataset(X_train_w, y_train_w)
    train_loader = torch.utils.data.DataLoader(
        train_dataset, batch_size=32, shuffle=False
    )

    for epoch in range(150):
        model.train()
        epoch_loss, num_b = 0.0, 0
        for bx, by in train_loader:
            optimizer.zero_grad()
            loss = criterion(model(bx), by)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()
            num_b += 1

        model.eval()
        with torch.no_grad():
            val_loss = criterion(model(X_val_w), y_val_w).item()

        history['train_loss'].append(epoch_loss / num_b)
        history['val_loss'].append(val_loss)

    wd_results[wd] = history
    print(f"Train: {history['train_loss'][-1]:.6f} | "
          f"Val: {history['val_loss'][-1]:.6f}")

# Ve bieu do
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for wd in weight_decays:
    label = f'wd={wd}' if wd > 0 else 'No decay'
    axes[0].plot(wd_results[wd]['train_loss'], label=label)
    axes[1].plot(wd_results[wd]['val_loss'], label=label)
```

```

axes[0].set_title('Train Loss theo Weight Decay', fontsize=13)
axes[1].set_title('Val Loss theo Weight Decay', fontsize=13)

for ax in axes:
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.legend()
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('weight_decay_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Bài tập 10: Nhận xét: weight decay quá lớn (10^{-2}) gây ra vấn đề gì?

19 Kỹ thuật 4: Khởi tạo trọng số (Weight Initialization)

19.1 Lý thuyết

Khởi tạo trọng số ảnh hưởng lớn đến quá trình hội tụ. Các phương pháp phổ biến:

- **Xavier/Glorot:** $W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right)$ — tốt cho Sigmoid/Tanh.
- **Kaiming/He:** $W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{in}}}\right)$ — tốt cho ReLU.
- **Orthogonal:** W là ma trận trực giao — rất tốt cho RNN, giúp gradient ổn định.
- **Zeros:** Tất cả trọng số = 0 — **RẤT TỆ**, mạng không học được.

19.2 Code thực hành

```

def init_weights(model, method='default'):
    """
    Khởi tạo trọng số theo phương pháp chỉ định.
    """
    for name, param in model.named_parameters():
        if 'weight_ih' in name or 'weight_hh' in name:
            if method == 'xavier':
                nn.init.xavier_uniform_(param)
            elif method == 'kaiming':
                nn.init.kaiming_uniform_(param, nonlinearity='relu')
            elif method == 'orthogonal':
                nn.init.orthogonal_(param)
            elif method == 'zeros':
                nn.init.zeros_(param)
            elif method == 'default':
                pass # Giữ nguyên khởi tạo mặc định của PyTorch
        elif 'bias' in name:
            nn.init.zeros_(param)
            # Meo: dat forget gate bias = 1 cho LSTM
            # (giúp LSTM nhớ thông tin lâu hơn o dau)
            if 'bias_ih' in name or 'bias_hh' in name:

```

```

        n = param.size(0)
        param.data[n//4 : n//2].fill_(1.0)

# === THI NGHIEM ===
init_methods = ['default', 'xavier', 'orthogonal', 'zeros']
init_results = {}

for method in init_methods:
    print(f"\n--- Init method: {method} ---")
    set_seed(42)

    model = RecurrentModel(
        model_type='LSTM', hidden_size=64, num_layers=2
    ).to(device)

    init_weights(model, method)

    history = train_model(
        model, X_train_w, y_train_w, X_val_w, y_val_w,
        num_epochs=100, learning_rate=0.001, batch_size=32,
        verbose=False
    )

    init_results[method] = history
    print(f"Val loss cuoi: {history['val_loss'][-1]:.6f}")

# Ve bieu do
fig, ax = plt.subplots(figsize=(10, 5))

for method in init_methods:
    ax.plot(init_results[method]['val_loss'], label=method, linewidth=2)

ax.set_title('Val Loss theo phuong phap khoi tao trong so', fontsize=14)
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('weight_init_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Bài tập 11:

1. Phương pháp “zeros” cho kết quả thế nào? Tại sao? (Gợi ý: symmetry problem.)
2. “Orthogonal” có tốt hơn “xavier” cho LSTM không?
3. Tại sao đặt forget gate bias = 1 lại giúp LSTM?

20 Kỹ thuật 5: Batch Normalization cho RNN

20.1 Lý thuyết

BatchNorm chuẩn hóa activation theo batch, giúp training ổn định hơn. Tuy nhiên, áp dụng cho RNN phức tạp hơn CNN. Cách phổ biến là dùng **LayerNorm** (chuẩn hóa theo features thay vì theo batch).

20.2 Code thực hành

```
class LSTMWithLayerNorm(nn.Module):
    """
    LSTM kết hợp LayerNorm trước lớp fully connected.
    LayerNorm chuẩn hóa output của LSTM, giúp training ổn định hơn.
    """
    def __init__(self, input_size=1, hidden_size=64, num_layers=2, dropout=0.0):
        super(LSTMWithLayerNorm, self).__init__()

        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0
        )

        # LayerNorm chuẩn hóa output của LSTM
        self.layer_norm = nn.LayerNorm(hidden_size)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        rnn_out, _ = self.lstm(x)
        last_output = rnn_out[:, -1, :]

        # Áp dụng LayerNorm
        normalized = self.layer_norm(last_output)
        prediction = self.fc(normalized)
        return prediction.squeeze(-1)

# === SO SANH: Có và không có LayerNorm ===
print("--- LSTM không có LayerNorm ---")
set_seed(42)
model_no_ln = RecurrentModel(
    model_type='LSTM', hidden_size=128, num_layers=3
).to(device)
history_no_ln = train_model(
    model_no_ln, X_train_w, y_train_w, X_val_w, y_val_w,
    num_epochs=100, learning_rate=0.002, verbose=False
)
print(f"Val loss cuối: {history_no_ln['val_loss'][-1]:.6f}")

print("\n--- LSTM có LayerNorm ---")
set_seed(42)
```



```

model_ln = LSTMWithLayerNorm(
    hidden_size=128, num_layers=3
).to(device)
history_ln = train_model(
    model_ln, X_train_w, y_train_w, X_val_w, y_val_w,
    num_epochs=100, learning_rate=0.002, verbose=False
)
print(f"Val loss cuoi: {history_ln['val_loss'][-1]:.6f}")

# Ve bieu do so sanh
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(history_no_ln['val_loss'], label='LSTM (khong LayerNorm)', linewidth=2)
ax.plot(history_ln['val_loss'], label='LSTM + LayerNorm', linewidth=2)
ax.set_title('Anh huong cua LayerNorm', fontsize=14)
ax.set_xlabel('Epoch')
ax.set_ylabel('Val Loss')
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('layernorm_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Bài tập 12: LayerNorm giúp training ổn định hơn không? Val loss có ít dao động hơn không?

21 Kỹ thuật 6: Early Stopping

21.1 Lý thuyết

Early stopping dừng training khi val loss không cải thiện sau một số epoch nhất định (*patience*). Đây là hình thức regularization đơn giản nhưng rất hiệu quả.

21.2 Code thực hành

```

class EarlyStopping:
    """
    Dừng training sớm khi val loss không cải thiện.

    patience: số epoch cho trước khi dừng
    min_delta: thay đổi tối thiểu được xem là cải thiện
    """
    def __init__(self, patience=10, min_delta=1e-5):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.should_stop = False
        self.best_model_state = None

    def __call__(self, val_loss, model):
        if self.best_loss is None:
            self.best_loss = val_loss
            self.best_model_state = model.state_dict().copy()

```

```

        elif val_loss < self.best_loss - self.min_delta:
            self.best_loss = val_loss
            self.counter = 0
            self.best_model_state = model.state_dict().copy()
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.should_stop = True

    def load_best_model(self, model):
        """Nap lai trong so tot nhat."""
        model.load_state_dict(self.best_model_state)

# === THI NGHIEM: Training voi Early Stopping ===
set_seed(42)
model = RecurrentModel(
    model_type='LSTM', hidden_size=128, num_layers=3
).to(device)

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
early_stopping = EarlyStopping(patience=15)

history = {'train_loss': [], 'val_loss': []}
train_dataset = torch.utils.data.TensorDataset(X_train_w, y_train_w)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32)

for epoch in range(300): # Dat max epoch lon, early stopping se dung som
    model.train()
    epoch_loss, nb = 0.0, 0
    for bx, by in train_loader:
        optimizer.zero_grad()
        loss = criterion(model(bx), by)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    nb += 1

    model.eval()
    with torch.no_grad():
        val_loss = criterion(model(X_val_w), y_val_w).item()

    history['train_loss'].append(epoch_loss / nb)
    history['val_loss'].append(val_loss)

# Kiem tra early stopping
    early_stopping(val_loss, model)

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1:3d} | Train: {epoch_loss/nb:.6f} "
              f"| Val: {val_loss:.6f} | Patience: "
              f"{early_stopping.counter}/{early_stopping.patience}")

    if early_stopping.should_stop:

```

```

print(f"\nEarly stopping tai epoch {epoch+1}!")
print(f"Val loss tot nhat: {early_stopping.best_loss:.6f}")
break

# Nap lai model tot nhat
early_stopping.load_best_model(model)
print("\nDa nap lai model co val loss tot nhat.")

```

Bài tập 13: Thay đổi patience thành 5, 10, 20, 30. Giá trị nào phù hợp nhất? Patience quá nhỏ có vấn đề gì? Quá lớn thì sao?

Phần VII

Tổng Hợp Và Bài Tập Báo Cáo

22 Bảng tổng hợp các kỹ thuật khắc phục

Vấn đề	Dấu hiệu	Kỹ thuật khắc phục	Tham số cần chỉnh
Overfitting	Val loss tăng trong khi train loss giảm	Dropout, Weight decay, Early stopping, Thêm dữ liệu	Dropout rate, λ , patience
Underfitting	Cả train và val loss đều cao	Tăng hidden size, thêm layers, giảm regularization, train lâu hơn	hidden_size, num_layers
Vanishing gradient	Gradient norm $\rightarrow 0$ ở layers đầu	LSTM/GRU thay RNN, Orthogonal init, LayerNorm	Chuyển sang LSTM/GRU
Exploding gradient	Loss = NaN/Inf, gradient norm rất lớn	Gradient clipping, giảm LR, LayerNorm	max_grad_norm, LR
Training chậm	Loss giảm rất chậm	Tăng LR, dùng scheduler, kiểm tra data	LR, batch_size

23 Bài tập tổng hợp nộp báo cáo

Yêu cầu: Sinh viên nộp báo cáo dạng Jupyter Notebook (.ipynb) hoặc PDF, bao gồm:

23.1 Phần 1: Cơ bản (6 điểm)

- (1 điểm) Lấy dữ liệu thành công từ cả 2 API (thời tiết và chứng khoán). Trình bày thống kê mô tả và biểu đồ khám phá dữ liệu.

2. **(1 điểm)** Tiền xử lý dữ liệu đúng: chuẩn hóa, tạo sliding window, chia train/val/test theo đúng thứ tự thời gian.
3. **(2 điểm)** Huấn luyện thành công 3 mô hình (RNN, LSTM, GRU) trên cả 2 bộ dữ liệu. Vẽ learning curves và prediction plots.
4. **(2 điểm)** Điền đầy đủ bảng tổng hợp kết quả thí nghiệm siêu tham số. Mỗi thí nghiệm có nhận xét ngắn gọn.

23.2 Phần 2: Nâng cao (4 điểm)

1. **(1 điểm)** Vẽ biểu đồ gradient norm, chỉ ra được vanishing gradient ở RNN.
2. **(1 điểm)** Áp dụng thành công ít nhất 3 kỹ thuật khắc phục (gradient clipping, dropout, weight decay, weight init, LayerNorm, early stopping).
3. **(1 điểm)** So sánh kết quả trước và sau khi áp dụng các kỹ thuật. Trình bày bằng bảng và biểu đồ.
4. **(1 điểm)** Viết phần kết luận: tổng hợp bài học kinh nghiệm, mô hình và cấu hình tốt nhất cho mỗi loại dữ liệu, hạn chế và hướng cải thiện.

23.3 Hướng dẫn trình bày

- Code phải chạy được từ đầu đến cuối không lỗi.
- Mỗi thí nghiệm cần có: mục tiêu, code, kết quả (bảng/biểu đồ), nhận xét.
- Biểu đồ phải có tiêu đề, nhãn trục, chú thích rõ ràng.
- Nhận xét phải cụ thể, dựa trên số liệu, không chung chung.

24 Tài liệu tham khảo

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chương 10: Sequence Modeling.
2. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8).
3. Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.
4. Karpathy, A. (2019). A Recipe for Training Neural Networks. Blog post.
5. PyTorch Documentation: <https://pytorch.org/docs/stable/nn.html#recurrent-layers>
6. Open-Meteo API: <https://open-meteo.com/en/docs>
7. Yahoo Finance API (yfinance): <https://pypi.org/project/yfinance/>